C++

PENG

Created: 20260817, Version: 20260831

Contents

1 Variables and Basic Types **1**
1.1 Primitive Built-in Types 1
1.1.1 Arithmetic Types 1
1.1.2 Type Conversions 1
1.1.3 Literals 2
1.2 Variables 2
1.2.1 Variable Definitions 2
1.2.2 Variable Declarations 2
1.2.3 Identifiers 2
1.2.4 Scope of a Name 2
1.3 Compound Types 3
1.3.1 References 3
1.3.2 Pointers 3

1 Variables and Basic Types

1.1 Primitive Built-in Types

1.1.1 Arithmetic Types

Definition 1.1 (Primitive builtin types).

- *arithmetic types:*
  - *integral types*
    - *characters:* `char`, `std::string`
    - *integers:* `int`*:* `signed` *or* `unsigned` *(*≥ 0*)*
    - *boolean values:* `bool`
  - *floating-point types:* `double`
- *void*

Remark 1.2. *Use* `double` *for floating-point computations.*

1.1.2 Type Conversions

Example 1.3 (Type conversions).

```
bool b = 20; // 0 for false and nonzero values for true
int i = b;   // true for 1 and false for 0
```

Example 1.4 (Signed and unsigned).

```
unsigned u = -1; // if 32-bit ints, 4294967295
int a = -1;
unsigned b = 1;
a * b // int value is converted to unsigned before addition
```

1.1.3 Literals

Definition 1.5 (Literals).
Integer literals:

- *decimal*
- *octal:* `0-`
- *hexadecimal:* `0x-`
- *suffix:*
  - `u`*: unsigned*
  - `L`*: long*

Floating-point literals (`double` *by default):*

- *decimal point*
- `E: 1E-2`
- *suffix:* `f/F`*: float*

character and string literals: `"Hello"` = `"Hello\0"`

Definition 1.6 (Escape sequences).

- *newline:* `\n`
- *horizontal tab:* `\t`
- *null:* `\0`

1.2 Variables

1.2.1 Variable Definitions

Remark 1.7.

- *Variables defined outside any function body are initialized to 0.*
- *Initialize every object of built-in type.*
- `std::cin >> int i;` *error, should be* `int i; std::cin >> i;`*.*

1.2.2 Variable Declarations

- File A define the variable `int ix;`. Other files: `extern int ix;`.
- All variables must be declared before being used.

1.2.3 Identifiers

Naming conventions:

- Variable name: lowercase
- Class name: uppercase

1.2.4 Scope of a Name

- Global scope: `::global_var`
- Block scope

Remark 1.8. *It is almost always a bad idea to define a local variable with the same name as a global variable that the function uses.*

1.3 Compound Types

1.3.1 References

- A reference is an alias.
- The type of a reference and the object must match exactly.

1.3.2 Pointers